

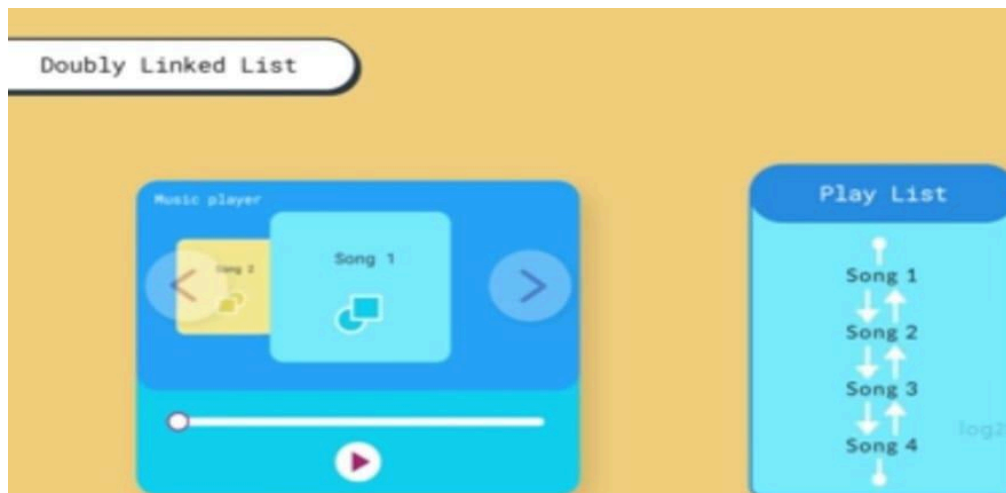
Doubly Linked Lists

Introduction

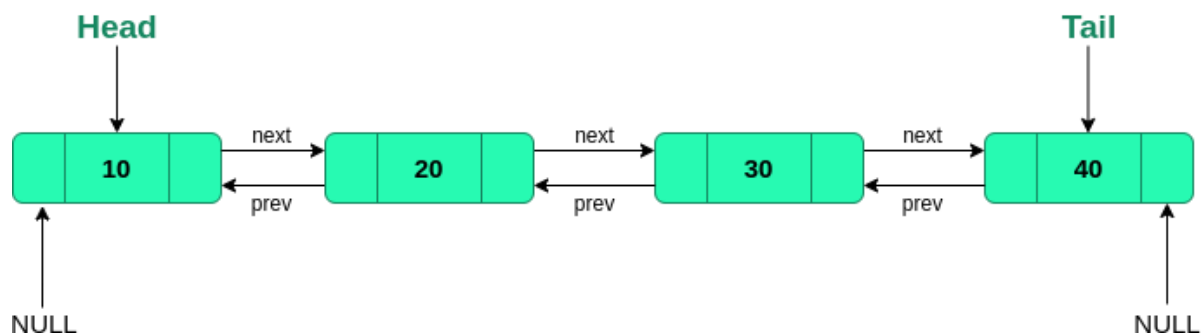
A **Doubly Linked List (DLL)** is a linear data structure that consists of nodes connected in a sequence, where each node contains three essential components:

1. **Data**: Stores the actual value or information
2. **Next**: A pointer/reference to the next node in the sequence
3. **Previous (Prev)**: A pointer/reference to the previous node in the sequence

This bidirectional linking structure distinguishes doubly linked lists from singly linked lists, enabling efficient traversal in both forward and backward directions. The ability to move in both directions makes doubly linked lists particularly useful for applications requiring frequent insertions, deletions, and bidirectional navigation.



Doubly Linked List



Structure and Components

Node Structure

Each node in a doubly linked list contains:

- **Data Field:** Holds the actual information (integer, string, object, etc.)
- **Next Pointer:** References the subsequent node in the list
- **Previous Pointer:** References the preceding node in the list

List Structure

The doubly linked list maintains:

- **Head Pointer:** Points to the first node in the list
- **Tail Pointer** (optional): Points to the last node for efficient rear operations
- **Size Counter** (optional): Tracks the number of nodes for quick length queries

Advantages of Doubly Linked Lists

Bidirectional Traversal

- Navigate forward from head to tail
- Navigate backward from tail to head
- Access previous elements without maintaining separate pointers

Efficient Operations

- **Insertion:** $O(1)$ time complexity when position is known
- **Deletion:** $O(1)$ time complexity when node reference is available
- **Reverse Traversal:** No need to reverse the entire list

Flexibility

- Dynamic size allocation
- Efficient memory utilization
- Easy implementation of complex data structures (dequeues, LRU caches)

Disadvantages of Doubly Linked Lists

Memory Overhead

- Additional memory required for previous pointers

- Approximately 33% more memory per node compared to singly linked lists

Implementation Complexity

- More complex pointer management
- Higher risk of pointer corruption
- Additional null pointer checks required

Cache Performance

- Poor spatial locality compared to arrays
- Random memory access patterns may impact performance

Operations on Doubly Linked Lists

1. Insertion Operations

Insert at Beginning (Prepend)

Process:

1. Create a new node with the given data
2. Set new node's next pointer to current head
3. Set current head's previous pointer to new node (if head exists)
4. Update head pointer to new node
5. Set new node's previous pointer to null

Time Complexity: $O(1)$ **Space Complexity:** $O(1)$

```
# Doubly Linked List Implementation
```

```
class Node:
```

```
    def __init__(self, data):
```

```
        self.data = data
```

```
        self.next = None
```

```
        self.prev = None
```

```
class doublell:
```

```
    def __init__(self):
```

```

        self.head = None

def addatbegin (self,data):

    newnode = Node(data)

    newnode.next = self.head

    if self.head != None:

        self.head.prev = newnode

    self.head = newnode

def display(self):

    if self.head == None:

        print("Linked list is empty")

    else:

        n = self.head

        while(n != None):

            print(n.data,"<->",end=" ")

            n = n.next

        print("Null")

dl = doublell()

dl.addatbegin(10)

dl.addatbegin(20)

dl.addatbegin(30)

dl.display()

```

Output:

30 <-> 20 <-> 10 <-> Null

Insert at End (Append)

Process:

1. Create a new node with the given data
2. If list is empty, make new node the head
3. Otherwise, traverse to the last node
4. Set last node's next pointer to new node
5. Set new node's previous pointer to last node
6. Set new node's next pointer to null

Time Complexity: $O(n)$ without tail pointer, $O(1)$ with tail pointer **Space Complexity:** $O(1)$

```
#Double linked list insert at end & Reverse linked list
```

```
class Node:
```

```
    def __init__(self, data):
```

```
        self.data = data
```

```
        self.next = None
```

```
        self.prev = None
```

```
class Linkedlist:
```

```
    def __init__(self):
```

```
        self.head = None
```

```
    def add_end(self, data):
```

```
        newnode = Node(data)
```

```
        if self.head is None:
```

```
            self.head = newnode
```

```
        else:
```

```

        n = self.head

        while n.next != None:

            n = n.next

        n.next = newnode

        newnode.prev = n

def display(self):

    if self.head == None:

        print("Linked list is empty")

    else:

        n = self.head

        while n != None:

            print(n.data,"<->",end=" ")

            n = n.next

        print("Null")

def revdisplay(self):

    n = self.head

    while n.next:

        n = n.next

    while n :

        print(n.data,"<->",end=" ")

        n = n.prev

    print("Null")

l = Linkedlist()

l.add_end(10)

```

```
l.add_end(20)

l.add_end(30)

l.display()

l.revdisplay()
```

Output:

```
10 <-> 20 <-> 30 <-> Null

30 <-> 20 <-> 10 <-> Null
```

Insert at Specific Position

Process:

1. Validate the position
2. Handle special cases (beginning, end)
3. Traverse to the desired position
4. Create new node and adjust pointers of adjacent nodes
5. Update new node's next and previous pointers

Time Complexity: $O(n)$ **Space Complexity:** $O(1)$

```
#double linked list insert at any position

class Node:

    def __init__(self,data):

        self.data = data

        self.next = None

        self.prev = None

class Linkedlist:

    def __init__(self):

        self.head = None

    def add_end(self,data):
```

```

newnode = Node(data)

if self.head is None:

    self.head = newnode

else:

    n = self.head

    while n.next != None:

        n = n.next

    n.next = newnode

    newnode.prev = n


def addatpos(self,pos,data):

    newnode = Node(data)

    if pos == 1:

        newnode.next = self.head

        self.head = newnode

        return

    n = self.head

    i = 1

    while n != None and i < pos - 1:

        n = n.next

        i += 1

    if n is None:

        print("Position out of range")

    else:

```



```

        newnode.next = n.next

        n.next = newnode

        if n.next is not None:

            n.next.prev = newnode

        n.next = newnode

def display(self):

    if self.head == None:

        print("Linked list is empty")

    else:

        n = self.head

        while n != None:

            print(n.data,"<->",end=" ")

            n = n.next

        print("Null")

l = Linklist()

l.add_end(10)

l.add_end(20)

l.add_end(30)

l.display()

l.addatpos(2,50)

l.display()

```

Output:

```
10 <-> 20 <-> 30 <-> Null
```

```
10 <-> 50 <-> 20 <-> 30 <-> Null
```

2. Deletion Operations

Delete from Beginning

Process:

1. Check if list is empty
2. Store reference to current head
3. Update head pointer to next node
4. Set new head's previous pointer to null (if exists)
5. Free memory of deleted node

Time Complexity: $O(1)$ **Space Complexity:** $O(1)$

```
#double linked list delete at begin
```

```
class Node:
```

```
    def __init__(self,data):
```

```
        self.data = data
```

```
        self.next = None
```

```
        self.prev = None
```

```
class Linkedlist:
```

```
    def __init__(self):
```

```
        self.head = None
```

```
    def add_end(self,data):
```

```
        newnode = Node(data)
```

```
        if self.head is None:
```

```
            self.head = newnode
```

```
        else:
```

```

        n = self.head

        while n.next != None:

            n = n.next

        n.next = newnode

        newnode.prev = n

def delatbegin(self):

    if self.head == None:

        print("Linked list empty")

    else:

        self.head = self.head.next

        self.head.prev = None

def display(self):

    if self.head == None:

        print("Linked list is empty")

    else:

        n = self.head

        while n != None:

            print(n.data,"<->",end=" ")

            n = n.next

        print("Null")

l = LinkedList()

```

```

l.add_end(10)

l.add_end(20)

l.add_end(30)

l.display()

l.delatbegin()

l.display()

```

Output:

```

10 <-> 20 <-> 30 <-> Null

20 <-> 30 <-> Null

```

Delete from End

Process:

1. Check if list is empty
2. If only one node, delete and update head
3. Otherwise, traverse to last node
4. Update second-last node's next pointer to null
5. Free memory of deleted node

Time Complexity: $O(n)$ without tail pointer, $O(1)$ with tail pointer **Space Complexity:** $O(1)$

```
#double linked list delete at end
```

```
class Node:
```

```
    def __init__(self,data):
```

```
        self.data = data
```

```
        self.next = None
```

```
        self.prev = None
```

```
class Linkedlist:
```

```
    def __init__(self):
```

```
        self.head = None
```

```

def add_end(self,data):

    newnode = Node(data)

    if self.head is None:

        self.head = newnode

    else:

        n = self.head

        while n.next != None:

            n = n.next

        n.next = newnode

        newnode.prev = n

def delatend(self):

    if self.head == None:

        print("Linked list is Empty")

    elif self.head.next == None:

        self.head = None

    else:

        n = self.head

        while n.next != None:

            n = n.next

        n.prev.next = None

def display(self):

    if self.head == None:

```

```

        print("Linked list is empty")

    else:

        n = self.head

        while n != None:

            print(n.data,"<->",end=" ")

            n = n.next

        print("Null")

l = Linkedlist()

l.add_end(10)

l.add_end(20)

l.add_end(30)

l.display()

l.delatend()

l.display()

```

Output:

```

10 <-> 20 <-> 30 <-> Null

10 <-> 20 <-> Null

```

Delete at any Value

Process:

1. Search for the node containing the target value
2. Once found, update adjacent nodes' pointers
3. Handle edge cases (first node, last node, single node)
4. Free memory of deleted node

Time Complexity: $O(n)$ **Space Complexity:** $O(1)$

```

#double linked list delete at any value

class Node:

    def __init__(self,data):

        self.data = data

        self.next = None

        self.prev = None

class Linkedlist:

    def __init__(self):

        self.head = None

    def add_end(self,data):

        newnode = Node(data)

        if self.head is None:

            self.head = newnode

        else:

            n = self.head

            while n.next != None:

                n = n.next

            n.next = newnode

            newnode.prev = n

    def delatvalue(self,x):

        if self.head is None:

            print("Linked list is empty")

```

```

        return

    if self.head.data == x:

        self.head = self.head.next

        return

    n = self.head

    while n.next != None:

        if n.data == x:

            if n.next != None:

                n.next.prev = n.prev #forward link -->

            if n.prev != None:

                n.prev.next = n.next #backward link <--

            return

        n = n.next

    print("Vlue not found")

def display(self):

    if self.head == None:

        print("Linked list is empty")

    else:

        n = self.head

        while n != None:

            print(n.data, "<->", end=" ")

            n = n.next

        print("Null")

```



```

l = LinkedList()

l.add_end(10)

l.add_end(20)

l.add_end(30)

l.display()

l.delatvalue(20)

l.display()

```

Output:

```

10 <-> 20 <-> 30 <-> Null

10 <-> 30 <-> Null

```

3. Traversal Operations

Forward Traversal

Process:

1. Start from head node
2. Visit each node and process its data
3. Move to next node using next pointer
4. Continue until reaching null

Time Complexity: $O(n)$ **Space Complexity:** $O(1)$

```

def display(self):

    if self.head == None:

        print("Linked list is empty")

    else:

        n = self.head

        while n != None:

```

```
print(n.data,"<->",end=" ")

n = n.next

print("Null")
```

Backward Traversal

Process:

1. Start from tail node (or traverse to end first)
2. Visit each node and process its data
3. Move to previous node using previous pointer
4. Continue until reaching null

Time Complexity: $O(n)$ **Space Complexity:** $O(1)$

```
def revdisplay(self):

    n = self.head

    while n.next:

        n = n.next

    while n :

        print(n.data,"<->",end=" ")

        n = n.prev

    print("Null")
```

Search Operation

Process:

1. Start from head (or tail for reverse search)
2. Compare each node's data with target value
3. Return node reference or position if found
4. Return null or -1 if not found

Time Complexity: $O(n)$ **Space Complexity:** $O(1)$

```

def search(self, x):

    # Find the tail node

    n = self.head

    if n is None:

        print("List is empty")

        return

    while n.next != None:

        n = n.next # move to tail

    # Now traverse backward

    while n != None:

        if n.data == x:

            print("True")

            return

        n = n.prev

    print("False")

```

Applications of Doubly Linked Lists



Browser History Management

Navigate forward and backward through web pages

Efficient insertion of new pages

Quick removal of pages from history



Music Player Playlists

Skip to next or previous songs

Insert songs at any position

Remove songs without affecting playback order



Operating System Process Management

Navigate through process queues

Efficient process scheduling

Bidirectional queue management



Text Editors

Undo/Redo functionality implementation

Cursor movement in both directions

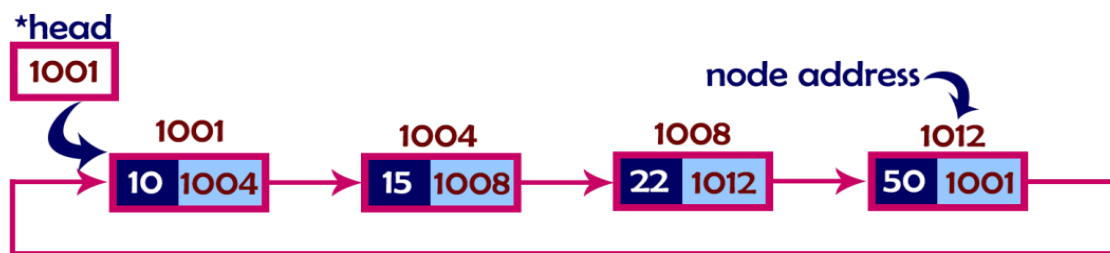
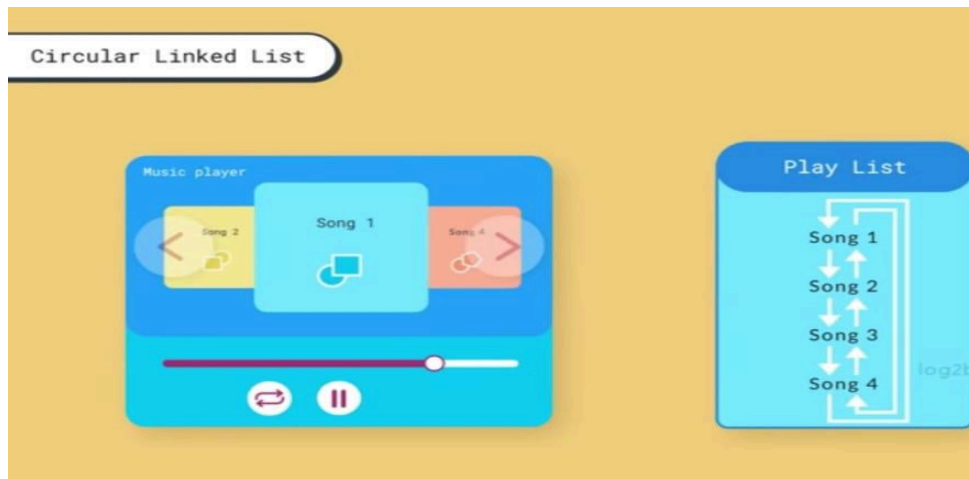
Efficient text insertion and deletion

Circular Linked Lists (CLL)

Introduction

A **Circular Linked List (CLL)** is a variation of a singly linked list where the **last node points back to the first node**, forming a circle. This structure allows for **continuous traversal** and is useful in applications requiring cyclic behavior, such as:

- Round-robin scheduling.
- Event loops.
- Playlists where elements are continuously cycled.



Basic Operations in CLL

1. **Insertion:**
 - o Insert a node at the **beginning**.
 - o Insert a node at the **end**.
2. **Deletion:**
 - o Delete a node from the **beginning**.
 - o Delete a node from the **end**.
3. **Traversal:**
 - o Display all elements in the list.

Insert a node at the end.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.ref = None
class linkedlist:
    def __init__(self):
        self.head = None

    # Add at end
    def add_end(self, data):
        newnode = Node(data)
        if self.head == None:
            self.head = newnode
            newnode.ref = newnode
        else:
            n = self.head
            while n.ref != self.head:
                n = n.ref
            n.ref = newnode
            newnode.ref = self.head

    def display(self):
        if self.head == None:
            print("Circular Linked List is empty")
        else:
            n = self.head
            while True:
                print(n.data, "-->", end=" ")
                n = n.ref
                if n == self.head:
                    break
            print("head")

l = linkedlist()
l.add_end(10)
l.add_end(20)
l.add_end(30)
l.display()
```

Output:

10 --> 20 --> 30 --> head

Insert a node at the beginning

```
class Node:
    def __init__(self, data):
        self.data = data
        self.ref = None
class linkedlist:
    def __init__(self):
        self.head = None

    # Add at beginning
    def add_begin(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            new_node.ref = new_node
        else:
            new_node.ref = self.head
            temp = self.head
            while temp.ref != self.head:
                temp = temp.ref
            temp.ref = new_node
            self.head = new_node

    def display(self):
        if self.head == None:
            print("Circular Linked List is empty")
        else:
            n = self.head
            while True:
                print(n.data, "-->", end=" ")
                n = n.ref
                if n == self.head:
                    break
            print("head")

l = linkedlist()
l.add_begin(10)
l.add_begin(20)
l.add_begin(30)
l.display()
```


Output:

30 --> 20 --> 10 --> head

Delete a node from the beginning.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.ref = None
class linkedlist:
    def __init__(self):
        self.head = None

    # Add at end
    def add_end(self, data):
        newnode = Node(data)
        if self.head == None:
            self.head = newnode
            newnode.ref = newnode
        else:
            n = self.head
            while n.ref != self.head:
                n = n.ref
            n.ref = newnode
            newnode.ref = self.head

    # Delete from beginning
    def delete_begin(self):
        if self.head is None:
            print("Circular Linked List is empty, nothing to delete")
        elif self.head.ref == self.head:
            self.head = None
        else:
            temp = self.head
            last = self.head
            while last.ref != self.head:
                last = last.ref
            self.head = self.head.ref
            last.ref = self.head

    def display(self):
        if self.head == None:
```

```

        print("Circular Linked List is empty")
    else:
        n = self.head
        while True:
            print(n.data, "-->", end=" ")
            n = n.ref
            if n == self.head:
                break
        print("head")

l = linkedlist()
l.add_end(10)
l.add_end(20)
l.add_end(30)
l.display()
l.delete_begin()
l.display()

```

Output:

10 --> 20 --> 30 --> head

20 --> 30 --> head

Delete a node from the end.

```

class Node:
    def __init__(self, data):
        self.data = data
        self.ref = None
class linkedlist:
    def __init__(self):
        self.head = None

    # Add at end
    def add_end(self, data):
        newnode = Node(data)
        if self.head == None:
            self.head = newnode
            newnode.ref = newnode
        else:
            n = self.head
            while n.ref != self.head:
                n = n.ref

```

```

        n.ref = newnode
        newnode.ref = self.head

# Delete from end
def delete_end(self):
    if self.head is None:
        print("Circular Linked List is empty, nothing to delete")
    elif self.head.ref == self.head:
        self.head = None
    else:
        temp = self.head
        while temp.ref.ref != self.head:
            temp = temp.ref
        temp.ref = self.head

def display(self):
    if self.head == None:
        print("Circular Linked List is empty")
    else:
        n = self.head
        while True:
            print(n.data, "-->", end=" ")
            n = n.ref
            if n == self.head:
                break
        print("head")

l = linkedlist()
l.add_end(10)
l.add_end(20)
l.add_end(30)
l.display()
l.delete_end()
l.display()

```

Output:

10 --> 20 --> 30 --> head

10 --> 20 --> head